

FRIDA MobileLabhacking

frida 1 :

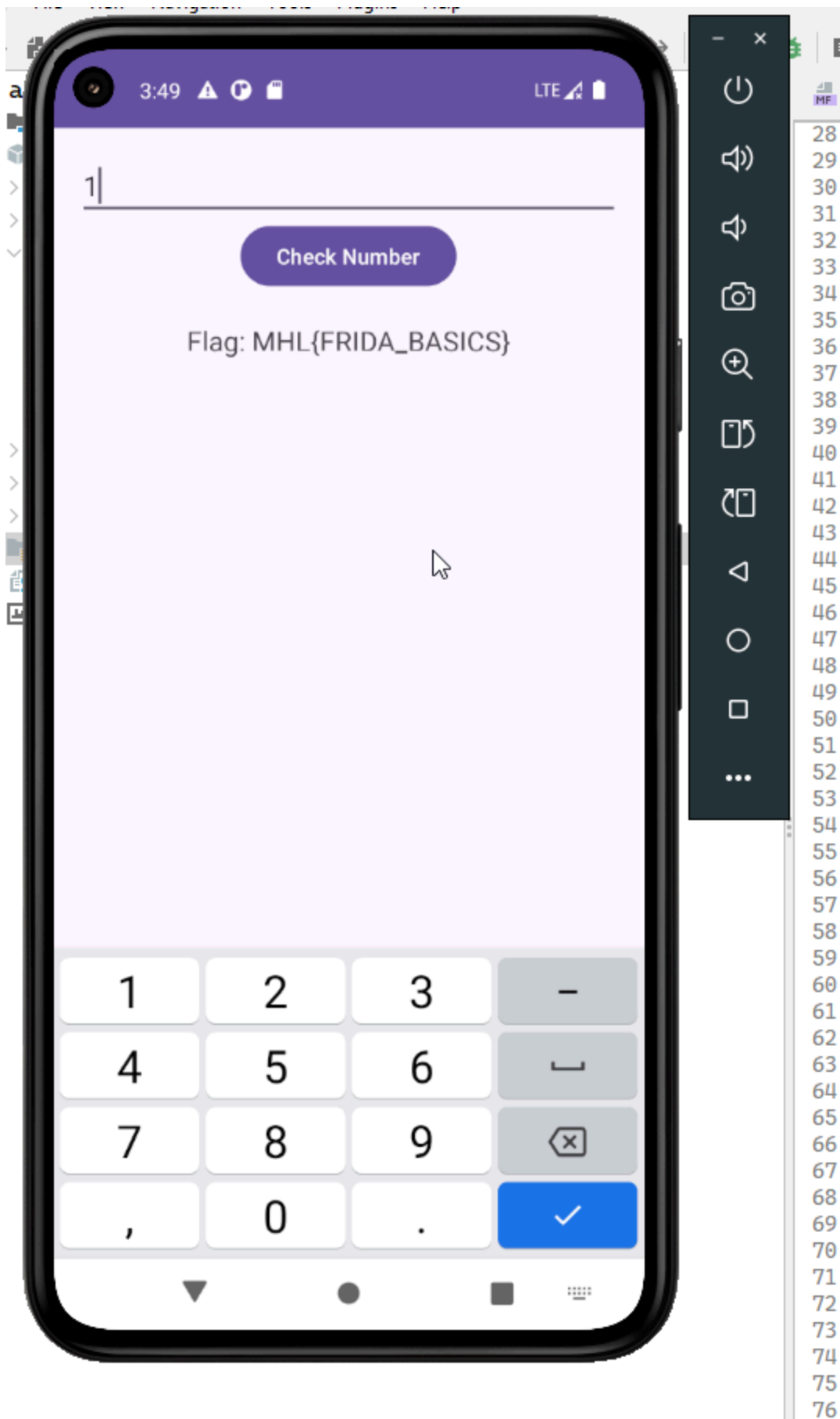
```
this.resultView = (TextView) view.findViewById();
this.randomNumber = generateRandomNumber();
submitBtn.setOnClickListener(new View.OnClickListener() { // from class: com.mobilehackinglab.f
    @Override // android.view.View.OnClickListener
    public final void onClick(View view) throws NumberFormatException {
        MainActivity.onCreate$lambda$0(numberInput, this, view);
    }
});
}

/* JADX INFO: Access modifiers changed from: private */
public static final void onCreate$lambda$0(EditText $numberInput, MainActivity this$0, View it) th
    Intrinsic.checkNotNullParameter(this$0, "this$0");
    TextView textView = null;
    try {
        int userNumber = Integer.parseInt($numberInput.getText().toString());
        if (userNumber == this$0.randomNumber) {
            byte[] bArrDecode = Base64.decode("TUhMe0ZSSURBX0JBU0lDU30=", 0);
            Intrinsic.checkNotNullExpressionValue(bArrDecode, "decode(...)");
            String decodedFlag = new String(bArrDecode, Charsets.UTF_8);
            TextView textView2 = this$0.resultView;
            if (textView2 == null) {
                Intrinsic.throwUninitializedPropertyAccessException("resultView");
                textView2 = null;
            }
            textView2.setText("Flag: " + decodedFlag);
            return;
        }
        TextView textView3 = this$0.resultView;
        if (textView3 == null) {
            Intrinsic.throwUninitializedPropertyAccessException("resultView");
            textView3 = null;
        }
        textView3.setText("Try again!");
    } catch (NumberFormatException e) {
        TextView textView4 = this$0.resultView;
        if (textView4 == null) {
            Intrinsic.throwUninitializedPropertyAccessException("resultView");
        } else {
            textView = textView4;
        }
        textView.setText("Please enter a valid number.");
    }
}

private final int generateRandomNumber() {
    return new Random().nextInt(999) + 1;
}
```

all we need is to hook this method and return fixed value that we will enter to check so they will be equal to each other revealing the flag

```
frida_mobilehacking.js > Java.perform() callback > implementation
1  Java.perform(function(){
2      var Main = Java.use('com.mobilehackinglab.fridaone.MainActivity');
3      Main.generateRandomNumber.implementation = function(){
4          return 1;
5      }
6  })
```



```

1  Java.perform(function(){
2      var Main = Java.use('com.mobilehackinglab.fridaone.MainActivity');
3      Main.generateRandomNumber.implementation = function(){
4
5          var result = this.generateRandomNumber();
6          console.log(result);
7          return 1;
8      }
9  })

```

Note we can observe the original return value and use it or just the approach i did previously

=====

Changing the value of static variable

```

/* JADX INFO: Access modifiers changed from: private */
public static final void onCreate$lambda$0(MainActivity this$0, View it) {
    Intrinsics.checkNotNullParameter(this$0, "this$0");
    if (Checker.INSTANCE.getCode() == 256) {
        Toast.makeText(this$0, "Congrats you got the flag", 1).show();
        byte[] bArrDecode = Base64.getDecoder().decode("TUhMe0lPRElGSUVEX1ZBUklBQkxkFfQ==");
        Intrinsics.checkNotNullExpressionValue(bArrDecode, "decode(...)");
        String decodedString = new String(bArrDecode, Charsets.UTF_8);
        TextView textView = this$0.tv;
        Button button = null;
        if (textView == null) {
            Intrinsics.throwUninitializedPropertyAccessException("tv");
            textView = null;
        }
        textView.setText(decodedString);
        Button button2 = this$0.btn;
        if (button2 == null) {
            Intrinsics.throwUninitializedPropertyAccessException("btn");
        } else {
            button = button2;
        }
    }
}

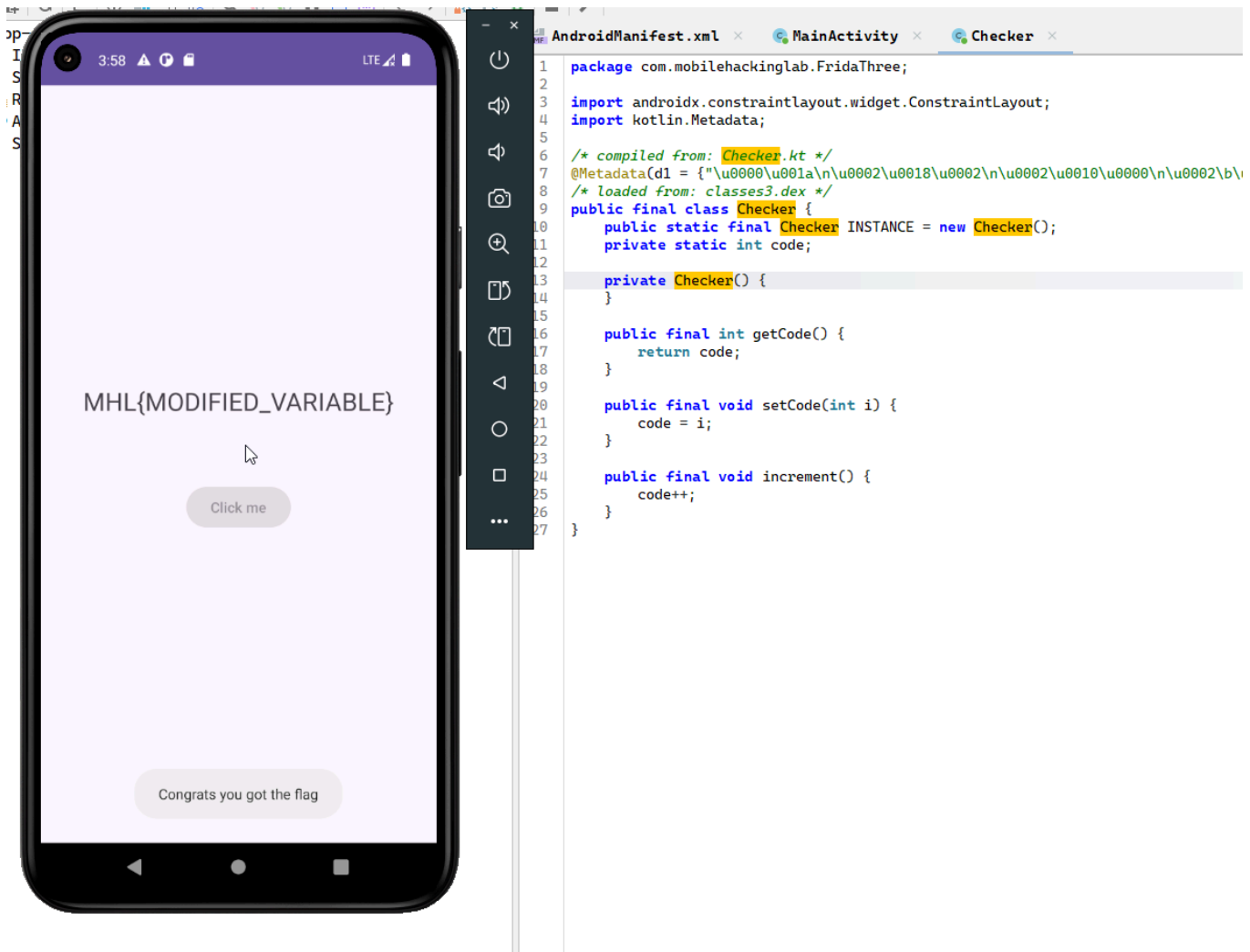
```

```

8  /* loaded from: classes3.dex */
9  public final class Checker {
10     public static final Checker INSTANCE = new Checker();
11     private static int code;
12
13     private Checker() {
14     }
15
16     public final int getCode() {
17         return code;
18     }
19
20     public final void setCode(int i) {
21         code = i;
22     }
23

```

The value from Checker.getCode() has to be 256 to get the flag



```
JS frida_mobilehacking.js > ...
1  Java.perform(function(){
2      var Checker = Java.use('com.mobilehackinglab.FridaThree.Checker');
3      Checker.getCode.implementation = function(){
4          ...
5          var result = this.getCode();
6          console.log(result);
7          return 256;
8      }
9  })
```

=====

Calling the method of non-static class

```
AndroidManifest.xml x MainActivity x Check x
1 package com.mobilehackinglab.FridaFour;
2
3 import androidx.constraintlayout.widget.ConstraintLayout;
4 import kotlin.Metadata;
5
6 /* compiled from: Check.kt */
7 @Metadata(d1 = {"\u0000\u0018\n\u0002\u0018\u0002\n\u0002\u0010\u0000\n\u0002\b\u0002\n\u0002\u0010\u0000e\n\u0000"},
8 /* loaded from: classes3.dex */
9 public final class Check {
10     public final String getFlag(int code) {
11         String result = "";
12         if (code == 1337) {
13             int length = "X]YnMZGPQJSYTRh".length();
14             for (int i = 0; i < length; i++) {
15                 char i2 = "X]YnMZGPQJSYTRh".charAt(i);
16                 StringBuilder sbAppend = new StringBuilder().append(result);
17                 int i3 = i2 ^ 21;
18                 if (i3 >= 0 && i3 <= 65535) {
19                     result = sbAppend.append((char) i3).toString();
20                 } else {
21                     throw new IllegalArgumentException("Invalid Char code: " + i3);
22                 }
23             }
24         }
25         return result;
26     }
27 }
```

So there is no call to this function getFlag()
so we need to create an instance and call the function

```
})
MHL{XORED_FLAG}
[Android Emulator 5554::PID::6690 ]-> █
```

=====

Calling the method from existing class :

why we could need to hook existing classes rather than creating an instance and using it ?

Creating an instance of `_MainActivity_` or any Android component directly using Frida can be tricky due to Android's lifecycle and threading rules. Android components, like Activity subclasses, rely on the application context for proper functioning. In Frida, we might lack the necessary context. Android UI components often require a specific thread with an associated `_Looper_`. If we're dealing with UI tasks, we need to ensure we're on the main thread with an active `_Looper_`. Activities are part of the larger Android application lifecycle. Creating an instance of `_MainActivity_` might need the app to be in

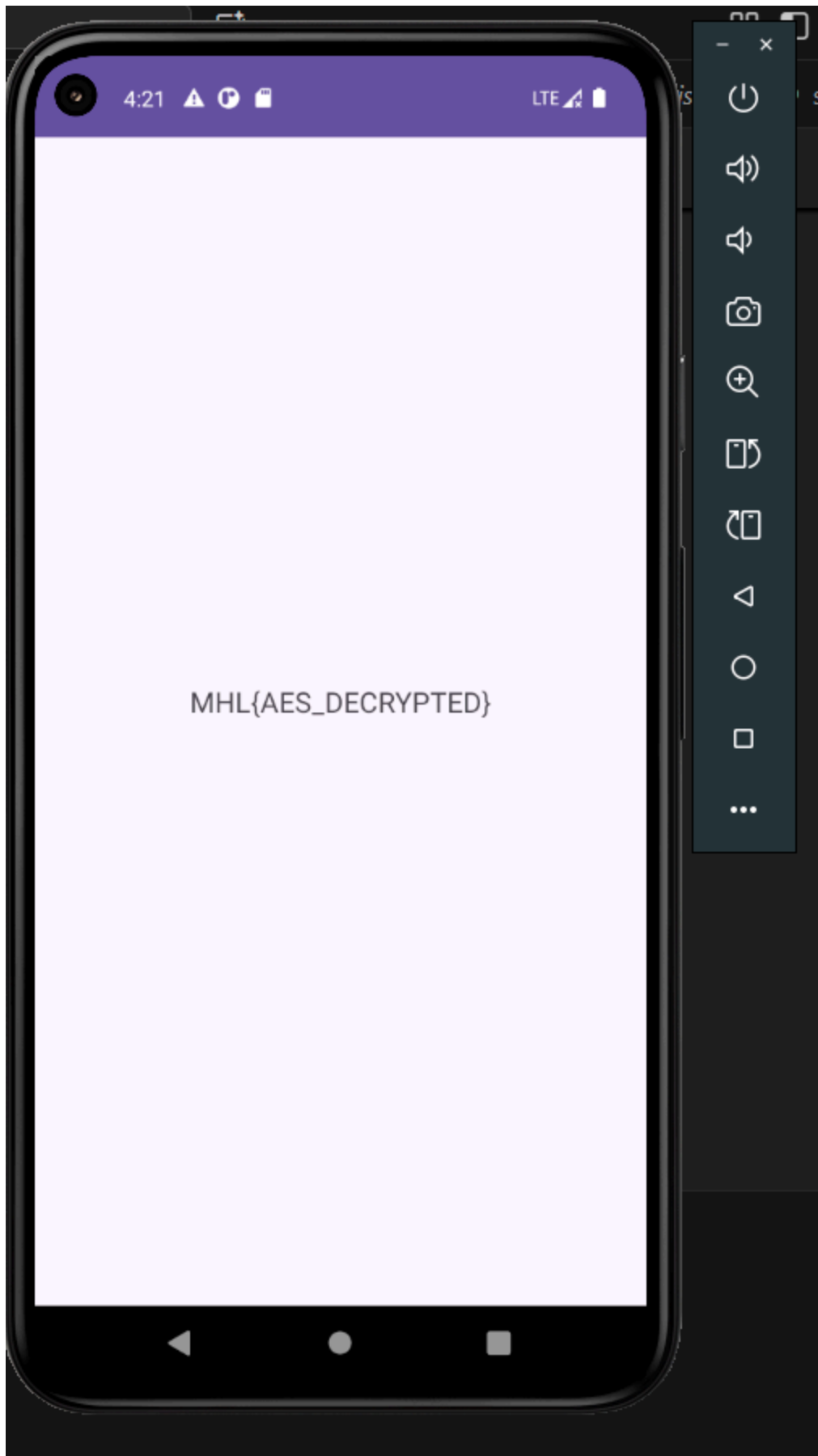
a specific state, and managing the entire lifecycle through Frida might not be straightforward. In conclusion, it's not a good idea to create an instance of `_MainActivity_`.

```
public final void flag(int key) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeException, NoSuchAlgorithmException {
    if (key == 1337) {
        byte[] iv = new byte[16];
        byte[] decodedBytes = Base64.getDecoder().decode("izhhERjfzRdbCdkqntOHkqPFFa12QxrYQDEAS3loYf8=");
        String paddedKey = StringsKt.padEnd("WILLIWONKESAWEL", 16, (char) 0);
        byte[] bytes = paddedKey.getBytes(Charsets.UTF_8);
        Intrinsics.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
        SecretKeySpec secretKeySpec = new SecretKeySpec(bytes, "AES");
        IvParameterSpec ivParameterSpec = new IvParameterSpec(iv);
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(2, secretKeySpec, ivParameterSpec);
        byte[] decryptedBytes = cipher.doFinal(decodedBytes);
        TextView t1 = getT1();
        Intrinsics.checkNotNull(decryptedBytes);
        t1.setText(new String(decryptedBytes, Charsets.UTF_8));
    }
}
```

Invoking methods on an existing instance can be easily done using Frida. For this, we will use two APIs:

- *Java.performNow* : Executes code within the context of the Java runtime.
- *Java.choose* : Enumerates through instances of the specified Java class (provided as the first argument) at runtime.

```
Java.performNow(function() {
    Java.choose('<Package>.<Class_Name>', {
        onMatch: function(instance) {
            // TODO
        },
        onComplete: function() {}
    });
});
```



=====

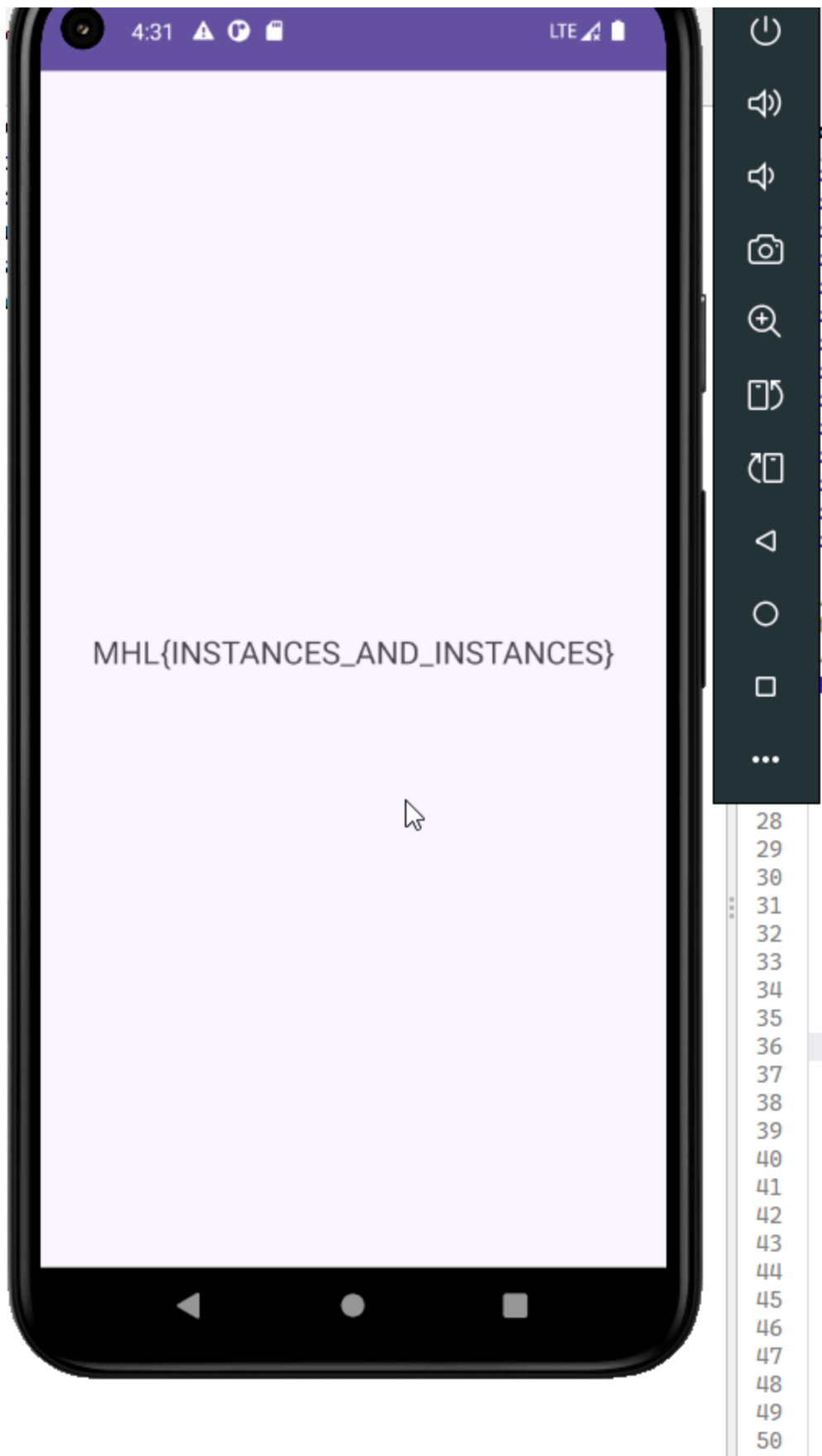
Passing an object as an argument :


```

        this.t1 = (TextView) view.findViewById(
            R.id.textView1);
        getFlag(new Checker());
    }

    public final void getFlag(Checker A) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeExcepti
        Intrinsic.checkNotNullParameter(A, "A");
        if ((1337 == A.getX()) & (120 == A.getY() / 10)) {
            Cipher cipher = Cipher.getInstance("AES");
            byte[] bytes = "MySecureKey12345".getBytes(Charsets.UTF_8);
            Intrinsic.checkNotNullExpressionValue(bytes, "getBytes(...)");
            SecretKeySpec secretKeySpec = new SecretKeySpec(bytes, "AES");
            cipher.init(2, secretKeySpec);
            byte[] decryptedBytes = Base64.decode("x6mSdprC1PuTsQ4MQk2AF1/6oxZXKVfCy6xtxMGigYo=", 0);
            byte[] bArrDoFinal = cipher.doFinal(decryptedBytes);
            Intrinsic.checkNotNullExpressionValue(bArrDoFinal, "doFinal(...)");
            String decrypted = new String(bArrDoFinal, Charsets.UTF_8);
            TextView textView = this.t1;
            if (textView == null) {
                Intrinsic.throwUninitializedPropertyAccessException("t1");
                textView = null;
            }
            textView.setText(decrypted);
        }
    }
}

```



Hook the existing main class and call the flag but to call the flag we need to find the checker instance and hook the getX and getY methods to return 1337 and 1200 to bypass the checks

=====

Hooking the constructure :

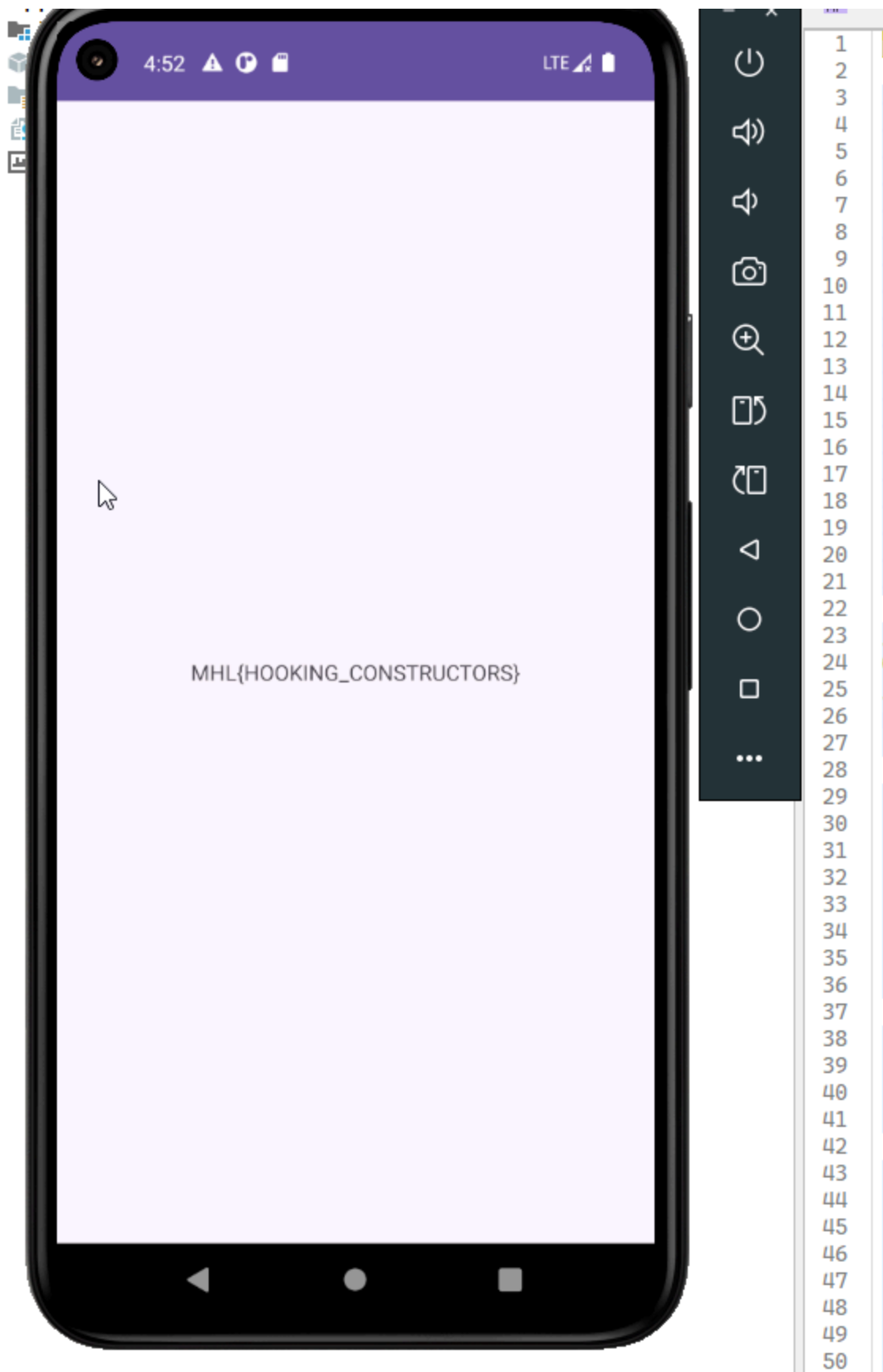
```
public final void getFlag(Checker ch) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeException, NoSuchAlgorithmException, Intrinsic.checkNotNullParameter(ch, "ch");
if (ch.getNum1() > 123 && ch.getNum2() > 321) {
    try {
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        byte[] decoded = Base64.getDecoder().decode("EQChhRX41M4SoqhRh1kRJV5KoywYwVMXMztKQVjJsBX75amRMzG7rWxWEysevLX");
        Intrinsic.checkNotNull(decoded);
        byte[] iv = ArraysKt.copyOfRange(decoded, 0, 16);
        byte[] encrypted = ArraysKt.copyOfRange(decoded, 16, decoded.length);
        byte[] bytes = "RANDOMKEY1234567".getBytes(Charsets.UTF_8);
        Intrinsic.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
        SecretKeySpec secretKey = new SecretKeySpec(bytes, "AES");
        cipher.init(2, secretKey, new IvParameterSpec(iv));
        byte[] decrypted = cipher.doFinal(encrypted);
        TextView t1 = getT1();
        Intrinsic.checkNotNull(decrypted);
        t1.setText(new String(decrypted, Charsets.UTF_8));
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

we need the checker class to return 123 for num1 and 321 for num2

but the main activity is calling the checker's constructure and passing 1 , 2

so we need to find the main class instance and inside the hooked class we need to call the flag method after hooking the checker class and setting the values we want

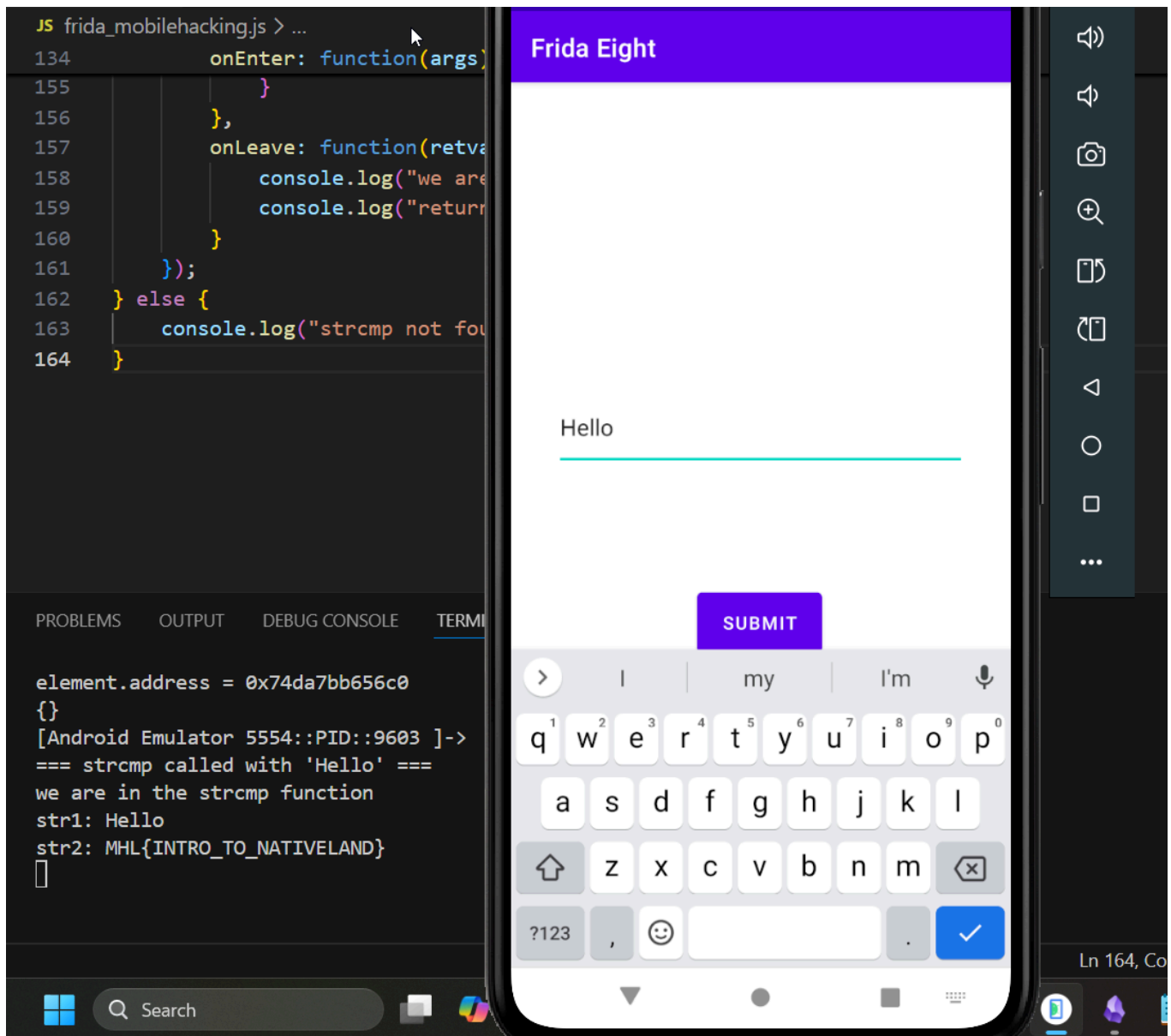
hooking the constructure --> class.\$init.implementation



=====

FRIDA Advanced :

1.Hooking Native methods using FRIDA



- `Java.cast()` expects a **Java object reference**, not a raw C pointer
- Native C strings (`char*`) are just memory addresses pointing to bytes in C memory space
- Java Strings are objects in the Java heap with a completely different structure
- You were trying to treat raw C memory as if it were a Java object structure

=====

Changing the return value of native function :

```
public final native int flag();
```

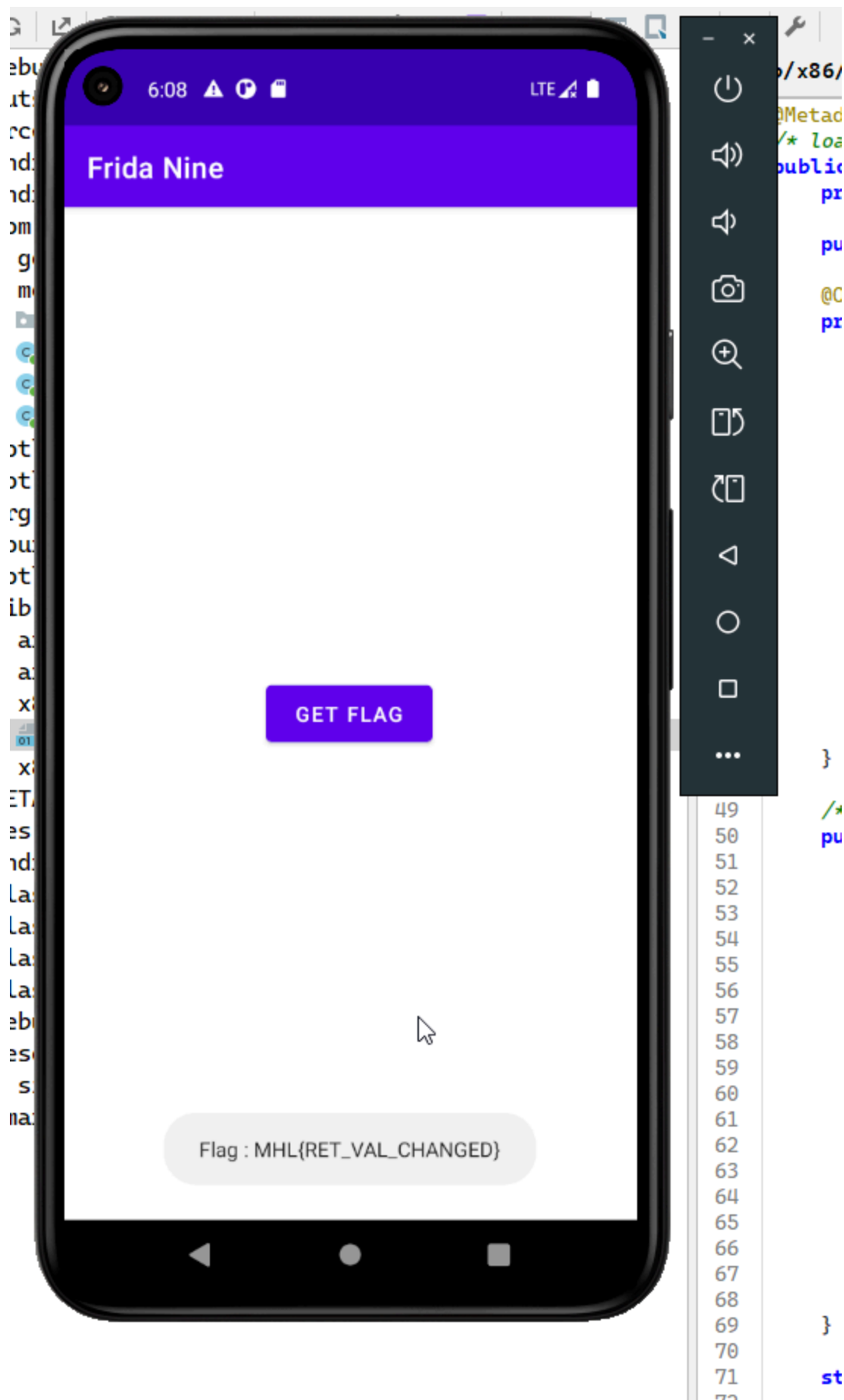
```
/* JADX INFO: Access modifiers changed from: private */
public static final void onCreate$lambda$0(MainActivity this$0, View it) throws BadPaddingException, NoSuchPaddingExceptio
    Intrinsic.checkNotNullParameter(this$0, "this$0");
    if (this$0.flag() == 623) {
        try {
            Cipher cipher = Cipher.getInstance("AES");
            cipher.init(2, this$0.getKey());
            byte[] decodedBytes = Base64.getDecoder().decode("ytmdCL/SVnkG+NqoHATEDsLW1xiXh0fev3RKLuy8VJo=");
            byte[] decryptedBytes = cipher.doFinal(decodedBytes);
            Intrinsic.checkNotNull(decryptedBytes);
            String flag = new String(decryptedBytes, Charsets.UTF_8);
            Toast.makeText(this$0, "Flag : " + flag, 1).show();
            return;
        } catch (Exception e) {
            e.printStackTrace();
            Toast.makeText(this$0, "Decryption failed: " + e.getMessage(), 1).show();
            return;
        }
    }
    Toast.makeText(this$0, "Better luck next time", 1).show();
}

static {
    System.loadLibrary("FridaNine");
}

private final SecretKeySpec getKey() {
    byte[] bytes = "1234567890123456".getBytes(Charsets.UTF_8);
    Intrinsic.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
}
```

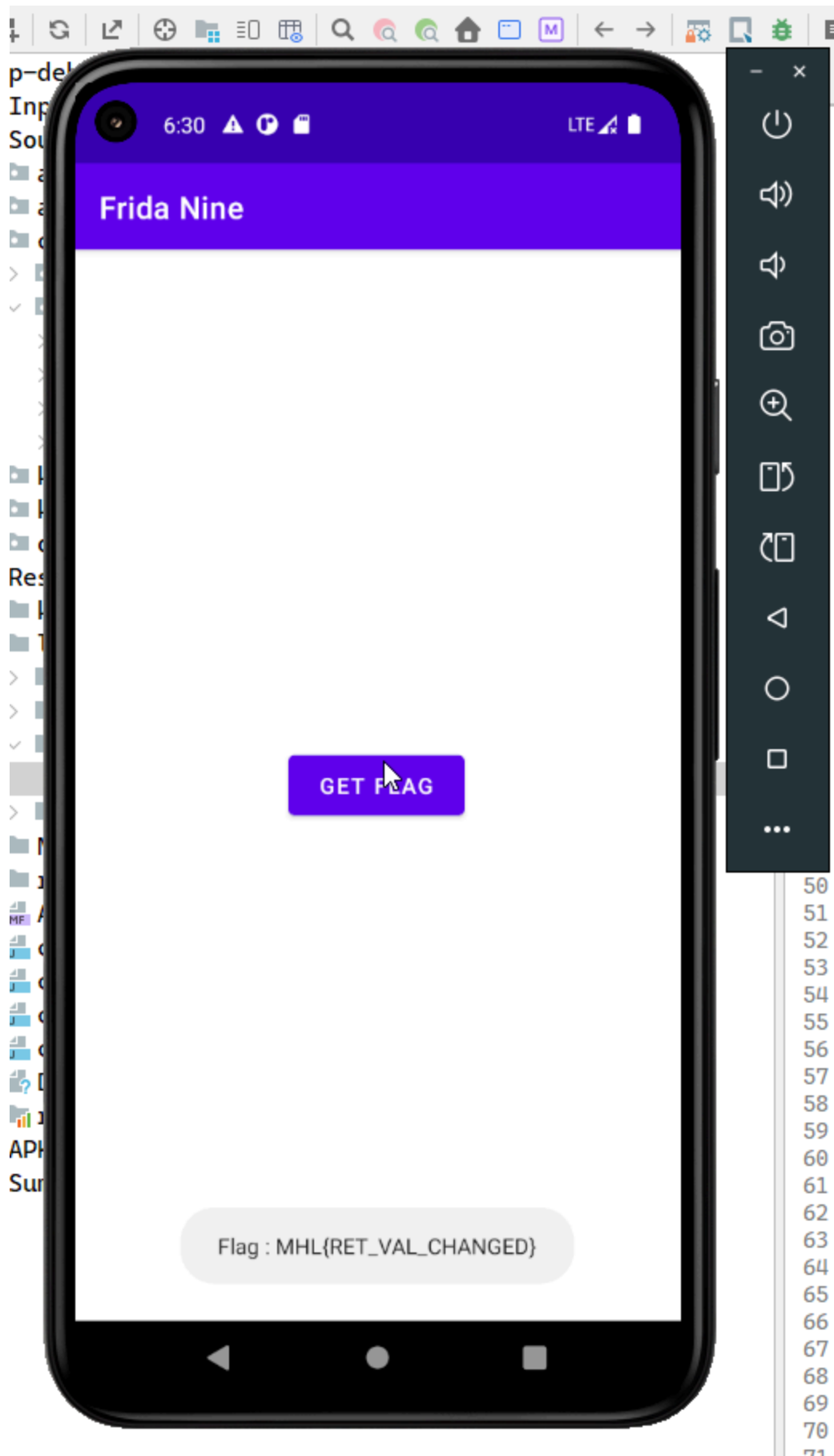
we can solve it the easy way by hooking the java side :

```
Java.perform(function(){
    var Main = Java.use('com.mobilehackinglab.FridaNine.Main');
    Main.flag.implementation = function(){
        return 623;
    }
});
```



OR from the native side for sure

```
1      },
2      onLeave:function(retval){
3          ...
4          retval.replace(623);
5          ...
6      }
7  })
8
```

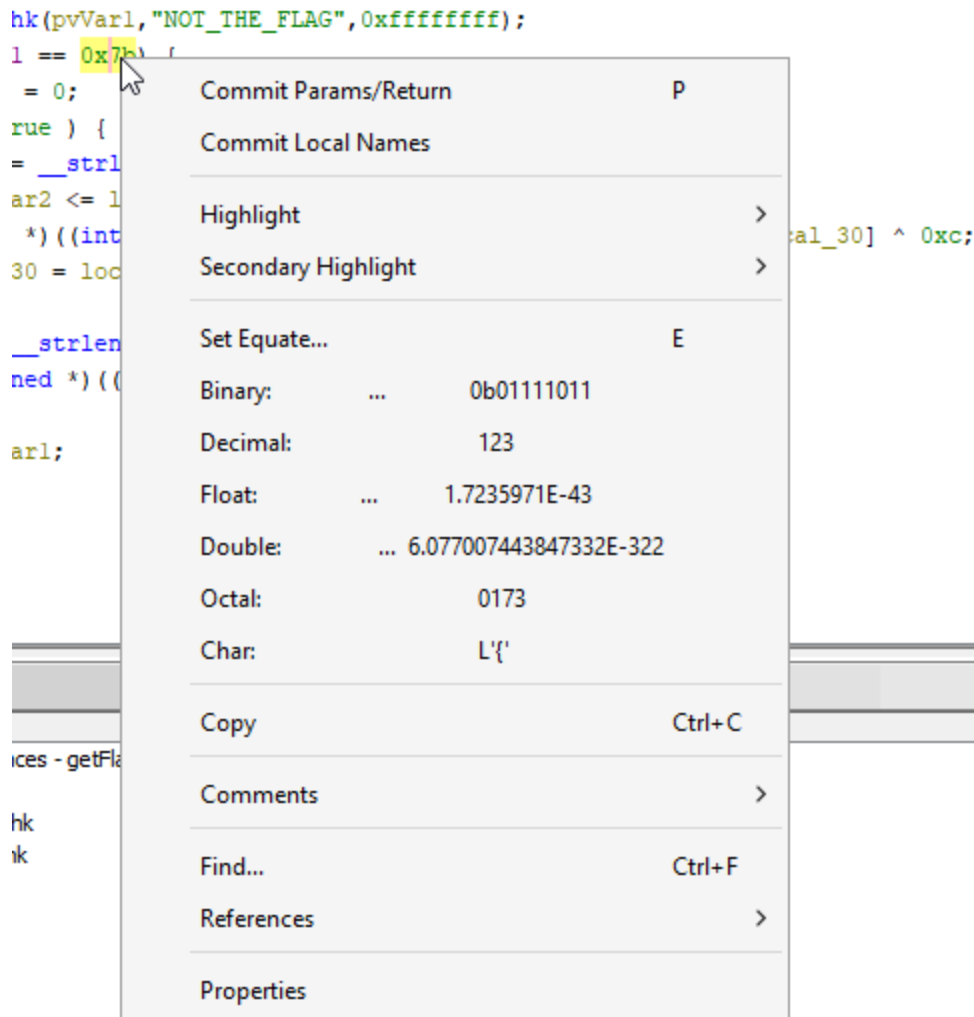



=====

Invoke Native Function :



```
1
2 /* getFlag(int) */
3
4 void * getFlag(int param_1)
5
6 {
7     void *pvVar1;
8     uint uVar2;
9     int iVar3;
10    uint local_30;
11
12    pvVar1 = malloc(100);
13    __strcpy_chk(pvVar1,"NOT_THE_FLAG",0xffffffff);
14    if (param_1 == 0x7b) {
15        local_30 = 0;
16        while( true ) {
17            uVar2 = __strlen_chk("AD@w_IO^IXSJYBOXECBq",0xffffffff);
18            if (uVar2 <= local_30) break;
19            *(byte *) ((int)pvVar1 + local_30) = "AD@w_IO^IXSJYBOXECBq"[local_30] ^ 0xc;
20            local_30 = local_30 + 1;
21        }
22        iVar3 = __strlen_chk("AD@w_IO^IXSJYBOXECBq",0xffffffff);
23        *(undefined *) ((int)pvVar1 + iVar3) = 0;
24    }
25    return pvVar1;
26 }
27
```



we need to invoke the getflag() function from the native library and pass 123 as argument

```
43  
44  
45 // Create NativeFunction: void* getFlag(int param_1)  
46 var getFlag = new NativeFunction(address, 'pointer', ['int']);  
47  
48 // Call with 0x7b (123 decimal) to get the real flag  
49 console.log("Calling getFlag(0x7b)...");  
50 var resultPtr = getFlag(0x7b);  
51  
52 // Read the string from the returned pointer  
53 var flag = resultPtr.readCString();  
54 console.log("Flag: " + flag);
```

```
// Read the string from the returned pointer
var flag = resultPtr.readCString();
console.log("Flag: " + flag);
Calling getFlag(0x7b)...
Flag: MHL{SECRET_FUNCTION}
[Android Emulator 5554::PID::10924 ]-> █ I
```

=====

Patch assembly instruction using FRIDA :

first we can solve it the easy way :

```
14  /* loaded from: classes3.dex */
15  public final class MainActivity extends AppCompatActivity {
16      private Button btn;
17      private TextView txt;
18
19      public final native String fetchFlag();
20
21      @Override // androidx.fragment.app.FragmentActivity, androidx.activity
```

it is not working here , seems the fetchFlag logic is not returning the flag

```
[Android Emulator 5554::PID::11268 ]-> Try Again
Try Again
Try Again
Try Again
█ I
```

so we need to reverse the native function and see what is happening exactly :

```
Decompile: Java_com_mobilehackinglab_FridaEleven_MainActivity_fetchFlag - (libFridaEleven.so)
1
2 /* WARNING: Removing unreachable block (ram,0x00010705) */
3 /* WARNING: Removing unreachable block (ram,0x00010721) */
4 /* WARNING: Removing unreachable block (ram,0x00010798) */
5 /* WARNING: Removing unreachable block (ram,0x00010765) */
6
7 undefined4 Java_com_mobilehackinglab_FridaEleven_MainActivity_fetchFlag(_JNIEnv *param_1)
8
9 {
10     undefined4 uVar1;
11     int in_GS_OFFSET;
12     char local_70 [100];
13     int local_c;
14
15     local_c = *(int *) (in_GS_OFFSET + 0x14);
16     __strcpy_chk(local_70,"Try Again",100);
17     uVar1 = _JNIEnv::NewStringUTF(param_1,local_70);
18     if (*(int *) (in_GS_OFFSET + 0x14) != local_c) {
19         /* WARNING: Subroutine does not return */
20         __stack_chk_fail();
21     }
22     return uVar1;
23 }
24
```

Disable the optimization option to display the never executed code
Then from the new output we can conclude that :

```
Decompile: Java_com_mobilehackinglab_FridaEleven_MainActivity_fetchFlag ...
7  undefined4 uVar3;
8  int in_GS_OFFSET;
9  uint local_98;
10 byte local_70 [100];
11 int local_c;
12
13 local_c = *(int *) (in_GS_OFFSET + 0x14);
14 if (true) {
15     __strcpy_chk(local_70,"Try Again",100);
16 }
17 else {
18     local_98 = 0;
19     while( true ) {
20         uVar1 = __strlen_chk("GBFqZK^IBU^BCYw",0xffffffff);
21         if (uVar1 <= local_98) break;
22         local_70[local_98] = "GBFqZK^IBU^BCYw"[local_98] ^ 10;
23         local_98 = local_98 + 1;
24     }
25     iVar2 = __strlen_chk("GBFqZK^IBU^BCYw",0xffffffff);
26     local_70[iVar2] = 0;
27 }
28 uVar3 = _JNIEnv::NewStringUTF(param_1, (char *)local_70);
29 if (*(int *) (in_GS_OFFSET + 0x14) == local_c) {
30     return uVar3;
31 }
32 /* WARNING: Subroutine does not return */
33 __stack_chk_fail();
34 }
35
```

The function fetchFlag() is always returning Try Again because of the hardcoded if condition to be always TRUE preventing the execution of the next block of code (else block) which execute the decryption logic

So from the logic we can see the encrypted text "GBFqZK^IBU^BCYw" that is XORed with the 10 which is 0x0A

So the flag can be constructed statically like that :

Char	ASCII	XOR 10	Result
G	0x47	0x4D	M
B	0x42	0x48	H
F	0x46	0x4C	L
q	0x71	0x7B	{
Z	0x5A	0x50	P
K	0x4B	0x41	A

Char	ASCII	XOR 10	Result
^	0x5E	0x54	T
I	0x49	0x43	C
B	0x42	0x48	H
U	0x55	0x5F	_
^	0x5E	0x54	T
B	0x42	0x48	H
C	0x43	0x49	I
Y	0x59	0x53	S
w	0x77	0x7D	}

Final decoded Flag :

MHL{PATCH_THIS}

OR we can patch the assembly instruction (the if condition which is always true) to be NOP so that the next instructions can be executed

That can be done by :

patching the binary (library)

Using FRIDA to patch the instructions in the runtime

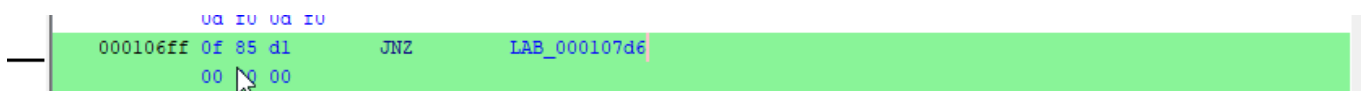
let's see how to use the X86Writer to solve our challenge :

```
var writer = new X86Writer(<address_of_the_instruction>);

try {
    // Insert instructions

    // Flush the changes to memory
    writer.flush();

} finally {
    // Dispose of the X86Writer to free up resources
    writer.dispose();
}
```



Memory.protect() -> change the permissions of a memory region